

Fase 0 — Lectura en frío (obligatoria y previa a cualquier herramienta)

Sebastián Quiceno Loaiza

Qué cree que hace el sistema y cuáles son sus entidades principales.

El sistema principalmente es un Sistema gestor de una farmacia, que tiene diferentes usuarios para manejarlo, el cual controla principalmente clientes, productos y sus ventas

Las entidades son:

- Persona (Clase Base): Clase abstracta usada como base para clientes y usuarios
- Cliente (Clase derivada de Persona): Entidad que representa a los clientes
- Usuario (Clase derivada de Persona): Entidad que representa al usuario controlador del sistema
- Producto (Clase Base): Clase abstracta para representar un producto
- Medicamento (clase derivada de Producto): Entidad que representa medicamento, es instanciable
- Medicamento capsula (clase derivada de Medicamento): Entidad que representa un medicamento de capsula
- Medicamento Liquido (clase derivada de Medicamento)
- Laboratorio: Entidad que solo guarda datos de un laboratorio (nombre, dirección y telefono)
- Movimiento: Entidad que representa una compra o movimiento, relaciona un cliente con un producto

Los tres lugares del código que, en su criterio, harían costoso un cambio, y por qué.

Analizando el código, los lugares mas complejos de cambiar serian los servicios, esto porque se encargan tanto de los casos de uso como de las operaciones CRUD. Los mas complejos de cambiar serian los que cargan los datos de los txt (ServicioCliente.cs, ServicioProductos.cs, ServicioUsuarios.cs). Estos son los mas costosos de cambiar porque son los que abarcan mas responsabilidades.

Otro lugar costoso de cambiar es la clase program.cs, la cual es la interfaz para el usuario. Si se quiere cambiar cualquier parte del código toca modificar el código existente. Aparte que se contacta directamente con las clases en vez de hacerlo a través de interfaces

Una pregunta que le haría al programador original si pudiera.

Si tiene algun archivo con la arquitectura usada, aparte de eso si pudiera explicar detalladamente su idea de negocio (para entender la arquitectura y refactorizarla)

NOTAS:

El sistema esta dividido en capas, dominio (Entidades), infraestructura (Servicios),

A simple vista tiene como entidades:

- Persona(Clase padre)
- Cliente
- Usuario
- Medicamento(Clase padre)
- Medicamento capsula
- Medicamento Liquido
- Laboratorio
- Movimiento
- Producto

ENTIDADES

Persona(Abstracta y padre):

Tiene nombre, cedula, telefono, correo. No tiene comprobaciones

Cliente(Derivada):

Tiene nombre, cedula, telefono, correo y Puntos.

Tiene método AcumularPuntos. POSIBLE VIOLACION SRP

Usuario(Derivada):

Tiene nombre, cedula, telefono, correo, **userName y password**

Laboratorio

Tiene nombre, direccion y teléfono (SOLO DATOS)

Producto(Abstracta Padre):

Tiene nombre, precio, stock, stockMinimo, FechaVencimiento

NO tiene Comprobaciones

Tiene método MostrarInformacion el cual usa Console.WriteLine (MALA PRACTICA), mejor sobrescribir método toString()

Medicamento(Clase derivada de producto)

Tiene nombre, precio, stock, stockMinimo, FechaVencimiento y laboratorio

Medicamento Capsula (Clase derivada de Medicamento) *tener en cuenta que Medicamento es instanciable*

Tiene nombre, precio, stock, stockMinimo, FechaVencimiento y **tipoRelleno(enum)**

Medicamento Liquido(Clase derivada de Medicamentos)

Tiene nombre, precio, stock, stockMinimo, FechaVencimiento, **MaterialEnvase(enum), mililitros**

Movimiento

Tiene fecha, cantidad, tipo(String, posible violación SRP), producto

Solo permite un tipo de producto por movimiento, no varios (REVISAR)

FABRICAS

ProductoFactory(Static)

Es una clase la cual se encarga de hacer medicamentos capsula y líquidos.

ESTA HARDCODEANDO ambos métodos:

- Capsula: stockMinimo, fechaVencimiento(6 meses), TipoRelleno
- Liquido: NO RECIBE MILILITROS; stockMinimo, fechaVencimiento (12 meses), MaterialEnvase (Vidrio)

SERVICIOS

ServicioCliente(No Implementa interfaz):

Se encarga de almacenar los clientes (Se debería encargar un repositorio) REVISAR METODO CARGAR (los carga del txt)

ServicioDescuento(Implementa interfaz IDescuento):

Solo retorna el valor con descuento

ServicioMovimiento():

Solo registra (agrega) los movimientos y los obtiene (Lo debería hacer a través de un repository)

ServicioNotificacion:

Solo se encarga de imprimir en consola una notificación(Mala practica)

ServicioProducto:

A diferencia de los demás la lista de productos es readonly

Tiene 2 eventos: StockMinimo y Vencimiento

Nuevamente se encarga de operaciones CRUD (se requiere de un repository)

Aparte de métodos CRUD tiene:

- VerificarStock: se encarga de disparar el evento si el stock esta bajo
- VerificarVencimiento: dispara el evento faltando 30 dias
- CargarDesdeArchivo: HardCodeo Laboratorio (Crea Medellin 4444444), aparte de declarar variables dentro de un ciclo (mala practica)

ServicioUsuario

Tiene lista de usuarios, mas de lo mismo, hace operaciones CRUD (Se requiere un repositorio)

Aparte de métodos CRUD tiene:

- Login: Usa directamente AspectoAutenticacion (Deberia ser por medio de una interfaz)